Description | Editorial | Solutions | Submissions

# 190. Reverse Bits

Solved

Easy | Topics | Companies

Reverse bits of a given 32 bits signed integer.

**Example 1:**

**Input:** n = 43261596

**Output:** 964176192

**Explanation:**

| Integer | Binary |
|---|---|
| 43261596 | 00000010100101000001111010011100 |
| 964176192 | 00111001011110000010100101000000 |

**Example 2:**

**Input:** n = 2147483644

**Output:** 1073741822

**Explanation:**

| Integer | Binary |
|---|---|
| 2147483644 | 01111111111111111111111111111100 |

5.9K | 213 | 67 Online

Code

Java | Auto

```java
class Solution {
    public int reverseBits(int n) {
      int res=0;
      for(int i=0;i<32;i++){
        res=(res<<1)|(n&1);
        n=n>>1;
      }return res;
    }
}
```

Saved | Ln 1, Col 1

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 | Case 2

Input

n =
43261596

Output

964176192

Expected

964176192

Leet | Accepted

All Submissions

Accepted 600 / 600 testcases passed

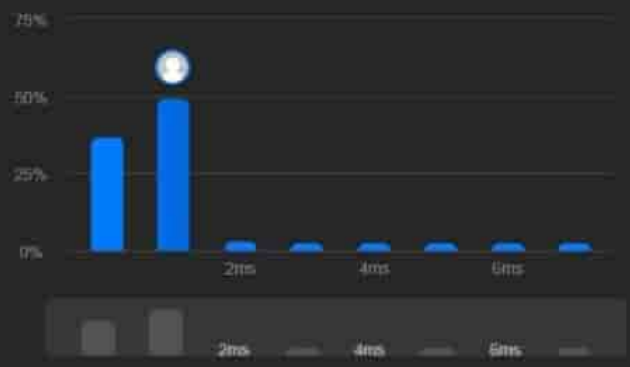
pavithra_e submitted at Mar 23, 2026 18:48

Runtime

1 ms | Beats 62.94%

Memory

42.28 MB | Beats 90.33%




Code | Java

```java
class Solution {
    public int reverseBits(int n) {
      int res=0;
      for(int i=0;i<32;i++){
        res=(res<<1)|(n&1);
        n=n>>1;
```

Description | Editorial | Solutions | Submissions

# 476. Number Complement

Solved

Easy | Topics | Companies

The **complement** of an integer is the integer you get when you flip all the `0`'s to `1`'s and all the `1`'s to `0`'s in its binary representation.

- For example, The integer `5` is `"101"` in binary and its **complement** is `"010"` which is the integer `2`.

Given an integer `num`, return *its complement*.

**Example 1:**

```
Input: num = 5
Output: 2
Explanation: The binary representation of 5 is 101
(no leading zero bits), and its complement is 010.
So you need to output 2.
```

**Example 2:**

```
Input: num = 1
Output: 0
Explanation: The binary representation of 1 is 1
(no leading zero bits), and its complement is 0. So
you need to output 0.
```

**Constraints:**

- $1 \le num < 2^{31}$

3.2K | 97 | 8 Online

Code

Java | Auto

```java
class Solution {
    public int findComplement(int num) {
        int mask=0;
        int temp=num;
        while(temp>0){
         mask=(mask<<1)|1;
         temp>>=1;
        } return num^mask;
    }
}
```

Saved | Ln 1, Col 1

Testcase | Test Result

**Accepted** Runtime: 0 ms

Case 1 | Case 2

Input

num =
5

Output

2

Expected

2

Accepted

All Submissions

**Accepted** 154 / 154 testcases passed

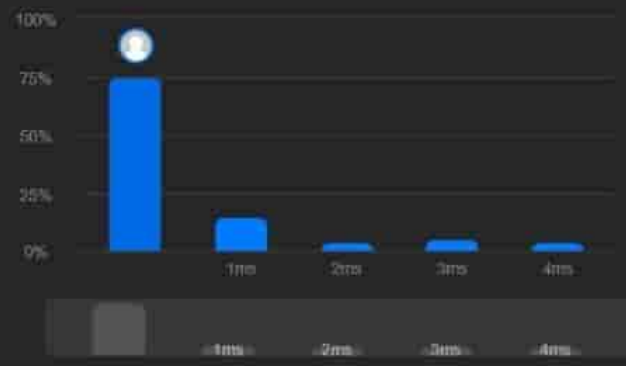
pavithra_e submitted at Mar 23, 2026 18:49

Runtime

0 ms | Beats 100.00%

Memory

42.08 MB | Beats 80.74%




Code | Java

```java
class Solution {
    public int findComplement(int num) {
        int mask=0;
        int temp=num;
        while(temp>0){
         mask=(mask<<1)|1;
```

Description | Editorial | Solutions | Submissions

# 326. Power of Three

Solved

Easy | Topics | Companies

Given an integer `n`, return `true` *if it is a power of three. Otherwise, return* `false`.

An integer `n` is a power of three, if there exists an integer `x` such that `n == 3^x`.

**Example 1:**

```
Input: n = 27
Output: true
Explanation: 27 = 3^3
```

**Example 2:**

```
Input: n = 0
Output: false
Explanation: There is no x where 3^x = 0.
```

**Example 3:**

```
Input: n = -1
Output: false
Explanation: There is no x where 3^x = (-1).
```

**Constraints:**

- $-2^{31} <= n <= 2^{31} - 1$

3.7K | 204 | 23 Online

Code

Java | Auto

```java
class Solution {
    public boolean isPowerOfThree(int n) {
     if(n<=0) return false;
     while(n%3==0){
        n=n/3;
     }   return  n==1;
    }
}
```

Saved | Ln 1, Col 1

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 | Case 2 | Case 3

Input

n =
27

Output

true

Expected

true

Leet | Accepted

All Submissions

Accepted 21040 / 21040 testcases passed

pavithra_e submitted at Mar 23, 2026 18:49

Runtime

8 ms Beats 92.04%

Memory

46.11 MB Beats 39.83%

Code Java

```java
class Solution {
    public boolean isPowerOfThree(int n) {
     if(n<=0) return false;
     while(n%3==0){
        n=n/3;
     }   return  n==1;
```

Description | Editorial | Solutions | Submissions

# 7. Reverse Integer

Solved

Medium | Topics | Companies

Given a signed 32-bit integer `x`, return `x` *with its digits reversed.* If reversing `x` causes the value to go outside the signed 32-bit integer range $[-2^{31}, 2^{31} - 1]$, then return `0`.

**Assume the environment does not allow you to store 64-bit integers (signed or unsigned).**

**Example 1:**

```
Input: x = 123
Output: 321
```

**Example 2:**

```
Input: x = -123
Output: -321
```

**Example 3:**

```
Input: x = 120
Output: 21
```

**Constraints:**

- $-2^{31} \le x \le 2^{31} - 1$

15.4K | 665 | 222 Online

Code

Java | Auto

```java
class Solution {
    public int reverse(int x) {
        int rev=0;
        int temp=x;

        while(temp!=0){
            int dig=temp%10;
            if(rev>214748364||rev<-214748364) return 0;
            rev=rev*10+dig;
            temp/=10;
        }
        return rev;
    }
```

Saved | Ln 1, Col 1

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 | Case 2 | Case 3

Input

x =
123

Output

321

Expected

321

Leet | Accepted

All Submissions

Accepted 1045 / 1045 testcases passed

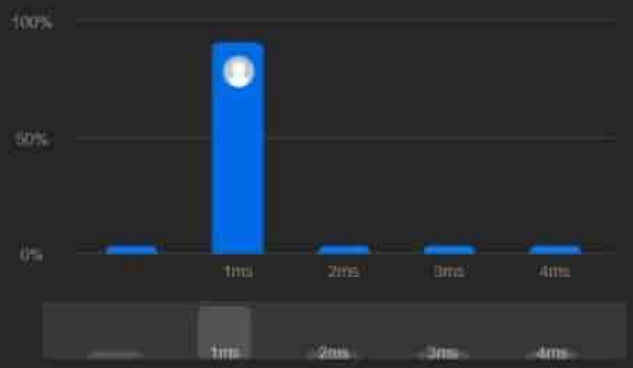
pavithra_e submitted at Mar 23, 2026 18:50

Runtime

1 ms | Beats 99.92%

Memory

42.63 MB | Beats 58.34%




Code | Java

```java
class Solution {
    public int reverse(int x) {
        int rev=0;
        int temp=x;

        while(temp!=0){
```

Description | Editorial | Solutions | Submissions

# 9. Palindrome Number

Solved

Easy | Topics | Companies | Hint

Given an integer `x`, return `true` *if* `x` *is a palindrome, and* `false` *otherwise.*

**Example 1:**

```
Input: x = 121
Output: true
Explanation: 121 reads as 121 from left to right and from right
to left.
```

**Example 2:**

```
Input: x = -121
Output: false
Explanation: From left to right, it reads -121. From right to
left, it becomes 121-. Therefore it is not a palindrome.
```

**Example 3:**

```
Input: x = 10
Output: false
Explanation: Reads 01 from right to left. Therefore it is not a
palindrome.
```

**Constraints:**

- $-2^{31} <= x <= 2^{31} - 1$

15.8K | 711 | 223 Online

Code

Java | Auto

```java
class Solution {
    public boolean isPalindrome(int x) {
        int rev=0;
        if(x<0) return false;
        int temp=x;
        while(temp!=0){
            int dig=temp%10;
            rev=rev*10+dig;
            temp/=10;
        }
        return (rev==x);
```

Saved | Ln 1, Col

Testcase | Test Result

Accepted 0 ms

Case 1 | Case 2 | Case 3

Input

x =
121

Output

true

Expected

true

Accepted

All Submissions

Accepted 11511 / 11511 testcases passed

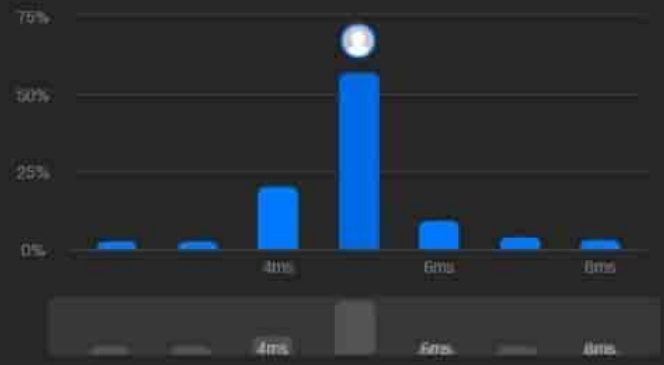
pavithra_e submitted at Mar 23, 2026 18:51

Runtime

5 ms | Beats 79.71%

Memory

46.19 MB | Beats 28.02%




Code | Java

```java
class Solution {
    public boolean isPalindrome(int x) {
        int rev=0;
        if(x<0) return false;
        int temp=x;
        while(temp!=0){
```

Description | Editorial | Solutions | Submissions

# 2652. Sum Multiples

Solved

Easy | Topics | Companies | Hint

Given a positive integer `n`, find the sum of all integers in the range `[1, n]` **inclusive** that are divisible by `3`, `5`, or `7`.

Return *an integer denoting the sum of all numbers in the given range satisfying the constraint.*

**Example 1:**

```
Input: n = 7
Output: 21
Explanation: Numbers in the range [1, 7] that are divisible by 3, 5, or 7 are 3, 5, 6, 7. The sum of these numbers is 21.
```

**Example 2:**

```
Input: n = 10
Output: 40
Explanation: Numbers in the range [1, 10] that are divisible by 3, 5, or 7 are 3, 5, 6, 7, 9, 10. The sum of these numbers is 40.
```

**Example 3:**

```
Input: n = 9
Output: 30
Explanation: Numbers in the range [1, 9] that are divisible by 3, 5, or 7 are 3, 5, 6, 7, 9. The sum of these numbers is 30.
```

593 | 35 | 8 Online

Code

Java | Auto

```java
class Solution {
    public int sumOfMultiples(int n) {
        int res=0;
        for(int i=1;i<=n;i++){
         if(i%3==0||i%5==0||i%7==0){
             res+=i;
         }
        } return res;
    }
}
```

Saved | Ln 1, Col 1

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 | Case 2 | Case 3

Input

n =
7

Output

21

Expected

21

Leet | Accepted

All Submissions

Accepted 1000 / 1000 testcases passed

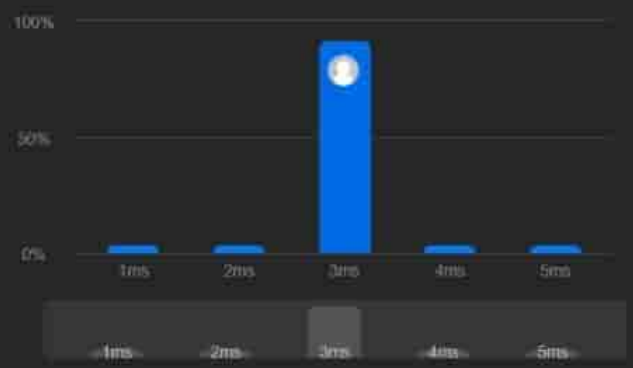
pavithra_e submitted at Mar 23, 2026 18:51

Runtime

3 ms | Beats 94.48%

Memory

42.71 MB | Beats 36.89%




Code | Java

```java
class Solution {
    public int sumOfMultiples(int n) {
        int res=0;
        for(int i=1;i<=n;i++){
         if(i%3==0||i%5==0||i%7==0){
             res+=i;
```

Description | Editorial | Solutions | Submissions

# 2739. Total Distance Traveled

Solved

Easy | Topics | Companies | Hint

A truck has two fuel tanks. You are given two integers, `mainTank` representing the fuel present in the main tank in liters and `additionalTank` representing the fuel present in the additional tank in liters.

The truck has a mileage of `10` km per liter. Whenever `5` liters of fuel get used up in the main tank, if the additional tank has at least `1` liters of fuel, `1` liters of fuel will be transferred from the additional tank to the main tank.

Return *the maximum distance which can be traveled.*

**Note:** Injection from the additional tank is not continuous. It happens suddenly and immediately for every 5 liters consumed.

**Example 1:**

```
Input: mainTank = 5, additionalTank = 10
Output: 60
Explanation:
After spending 5 litre of fuel, fuel remaining is
(5 - 5 + 1) = 1 litre and distance traveled is
50km.
After spending another 1 litre of fuel, no fuel
gets injected in the main tank and the main tank
becomes empty.
Total distance traveled is 60km.
```

327 | 22 | 7 Online

Code

Java | Auto

```java
class Solution {
    public int distanceTraveled(int mainTank, int additionalTank) {
        int distance=0;
        while(mainTank>0){
            mainTank--;
            distance+=10;
            if(distance%50==0&&additionalTank>0){
                mainTank++;
                additionalTank--;
            }}return distance;
    }
}
```

Saved | Ln 1, Col 1

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 | Case 2

Input

mainTank =
5

additionalTank =
10

Output

60

Leet | Accepted

All Submissions

Accepted 10000 / 10000 testcases passed

pavithra_e submitted at Mar 23, 2026 18:52

Runtime

5 ms | Beats 36.97%

Memory

45.52 MB | Beats 98.02%

Code | Java

```java
class Solution {
    public int distanceTraveled(int mainTank, int
        int distance=0;
        while(mainTank>0){
            mainTank--;
            distance+=10;
```

Description | Editorial | Solutions | Submissions

# 1518. Water Bottles

Solved

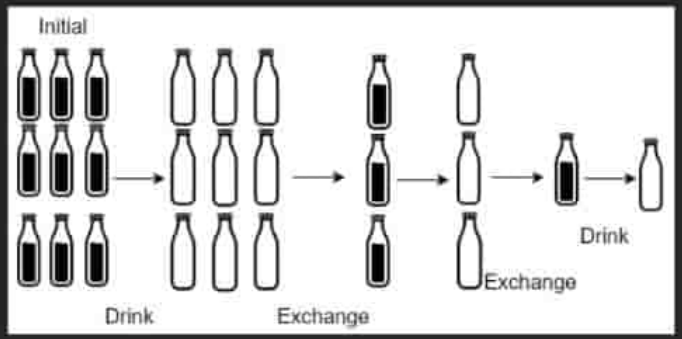
Easy | Topics | Companies | Hint

There are `numBottles` water bottles that are initially full of water. You can exchange `numExchange` empty water bottles from the market with one full water bottle.

The operation of drinking a full water bottle turns it into an empty bottle.

Given the two integers `numBottles` and `numExchange`, return *the **maximum** number of water bottles you can drink.*

**Example 1:**




```
Input: numBottles = 9, numExchange = 3
Output: 13
Explanation: You can exchange 3 empty bottles to
get 1 full water bottle.
Number of water bottles you can drink: 9 + 3 + 1 =
```

2.3K | 257 | 17 Online

Code

Java | Auto

```java
class Solution {
    public int numWaterBottles(int numBottles, int numExchange) {
        int drank=numBottles;
        int empty=numBottles;
        while(empty>=numExchange){
            int newbot=empty/numExchange;
            drank+=newbot;
            empty=newbot+(empty%numExchange);
        }
        return drank;
    }
}
```

Saved | Ln 1, Col 1

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 | Case 2

Input

numBottles =
9

numExchange =
3

Output

13

Leet | Accepted

All Submissions

Accepted 70 / 70 testcases passed

pavithra_e submitted at Mar 23, 2026 18:53

Runtime

0 ms | Beats 100.00%

Memory

42.12 MB | Beats 52.96%

Code | Java

```java
class Solution {
    public int numWaterBottles(int numBottles, int
        int drank=numBottles;
        int empty=numBottles;
        while(empty>=numExchange){
            int newbot=empty/numExchange;
```

Description | Editorial | Solutions | Submissions

# 1952. Three Divisors

Solved

Easy | Topics | Companies | Hint

Given an integer `n`, return `true` *if* `n` *has* ***exactly three positive divisors****. Otherwise, return* `false`.

An integer `m` is a **divisor** of `n` if there exists an integer `k` such that `n = k * m`.

**Example 1:**

```
Input: n = 2
Output: false
Explantion: 2 has only two divisors: 1 and 2.
```

**Example 2:**

```
Input: n = 4
Output: true
Explantion: 4 has three divisors: 1, 2, and 4.
```

**Constraints:**

- $1 <= n <= 10^4$

GoDaddy Get email that looks as professional as your business is.

634 | 29 | 7 Online

Code

Java | Auto

```java
class Solution {
    public boolean isThree(int n) {
      int count=0;
      for(int i=1;i<=n;i++){
        if(n%i==0){
            count++;
        }
      }if(count==3) return true;
      return false;
    }
}
```

Saved | Ln 1, Col 1

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 | Case 2

Input

n =
2

Output

false

Expected

false

Leet | Accepted

All Submissions

Accepted 229 / 229 testcases passed

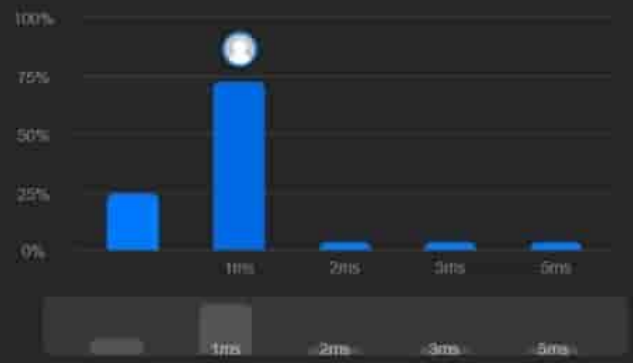
pavithra_e submitted at Mar 23, 2026 18:53

Runtime

1 ms | Beats 75.40%

Memory

42.20 MB | Beats 68.81%




Code | Java

```java
class Solution {
    public boolean isThree(int n) {
      int count=0;
      for(int i=1;i<=n;i++){
        if(n%i==0){
            count++;
```

Description | Editorial | Solutions | Submissions

# 397. Integer Replacement

Solved

Medium | Topics | Companies

Given a positive integer `n`, you can apply one of the following operations:

1. If `n` is even, replace `n` with `n / 2`.
2. If `n` is odd, replace `n` with either `n + 1` or `n - 1`.

Return *the minimum number of operations needed for* `n` *to become* `1`.

**Example 1:**

```
Input: n = 8
Output: 3
Explanation: 8 -> 4 -> 2 -> 1
```

**Example 2:**

```
Input: n = 7
Output: 4
Explanation: 7 -> 8 -> 4 -> 2 -> 1
or 7 -> 6 -> 3 -> 2 -> 1
```

**Example 3:**

```
Input: n = 4
Output: 2
```

1.4K | 36 | 7 Online

Code

Java | Auto

```java
class Solution {
    public int integerReplacement(int n) {
        long num = n;
        int steps = 0;
        while (num != 1) {
            if (num%2==0){
                num=num/2;
            } else {
                if (num==3||num%4==1) {
                    num--;} else {
                    num++;}}
            steps++;
        }return steps;
```

Saved | Ln 1, Col 1

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 | Case 2 | Case 3

Input

n =
8

Output

3

Expected

3

Leet | Accepted

All Submissions

Accepted 47 / 47 testcases passed

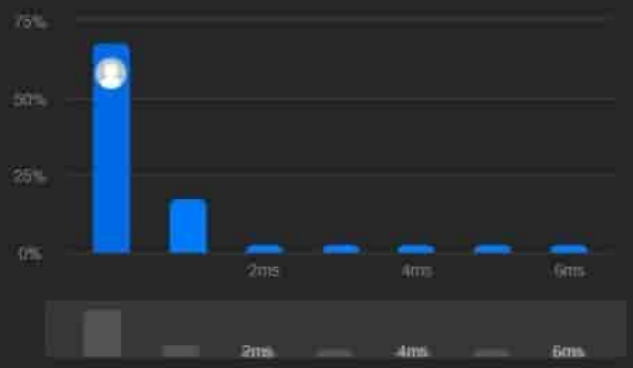
pavithra_e submitted at Mar 23, 2026 18:54

Runtime

0 ms Beats 100.00%

Memory

42.03 MB Beats 69.24%




Code Java

```java
class Solution {
    public int integerReplacement(int n) {
        long num = n;
        int steps = 0;
        while (num != 1) {
            if (num%2==0){
```